实 验 报 告

实验名称 网贷平台的风控建模与调优

课程名称 网络数据风控技术

专业班级：2023 级数据科学与大数据技术专业

学生姓名： 刘彦 学 号：2352018

指导教师： 叶晨 成 绩：

实验日期：2025.11.10-2025.11.18

同济大学

一、 实验背景

信用贷（原蚂蚁借呗）是支付宝平台提供的小额信用贷款服务。互联网小额信用贷款平台（简称网贷平台）根据风险控制和准入标准对支付宝实名用户进行筛选，为筛选出的优质客户提供不同的借款额度。网贷平台的快速发展得益于互联网平台建立了自建的场景流量信用评分体系，形成了获客和贷款的闭环体验。

本实验需要通过约 400 个数据维度评估用户当前的信用状态，结合脱敏处理过的样本数据，对贷款风险进行建模，预测用户 6 个月内的逾期率，为平台提供关键的决策依据。

二、 问题分析与数据准备（10%）

1. 问题分析

核心任务是构建一个信用贷款违约预测模型。这本质上是一个二元分类问题，即根据借款人的多维度信息（个人特征、网络行为、第三方数据等），预测其是否会违约（target=1）。

建模思路整体流程遵循标准的机器学习建模范式。首先，将多个数据源进行整合和特征工程，构建出一个包含 m 个特征的宽表。然后，将数据划分为训练集（X_train, y_train）和验证集（X_valid, y_valid），用于训练和调优一个映射函数 F。最终，该模型将应用于测试集 X_test 以生成预测结果。

模型性能的最终评价指标是 AUC（ROC 曲线下面积）。ROC 曲线以假正例率（FPR）为横轴，真正例率（TPR）为纵轴：

$$FPR = \frac{FP}{FP + TN}, TPR = \frac{TP}{TP + FN}$$

$$AUC = \sum_{i=1}^{n-1} \frac{(TPR_{i+1} + TPR_i) \times (FPR_{i+1} - FPR_i)}{2} = \frac{\sum_{i=1}^{m} \sum_{i=1}^{n} I(p_i > p_j)}{m \times n}$$

其中，p_i是第 i 个正样本的预测概率，p_j第 j 个负样本的预测概率，$I(\cdot)$是指示函数，当条件为真时返回 1，否则返回 0。

该指标的特点是衡量模型对正负样本的排序能力，即正确预测“违约”的概率高于“正常还款”的概率，而对具体的预测概率阈值不敏感，这非常适合于风险排序场景。

2. 数据准备

本次实验的数据集包含三个部分，分别是借款人信息表（Master_*_Set）、用户更新信息表（Userupdate_Info_*_Set）和用户行为日志表（LogInfo_*_Set）。这些数据共同描述了借款人的静态信息、信息修改历史以及平台使用行为。所有文件均为 GBK 编码。

Master_*_Set（借款人信息表）：该表是主数据表，包含了每位借款人的详细背景信息，涵盖了个人、工作、社交网络等多个维度。包含 30,000 条记录和 228 个字段。主要由数值型（float64 和 int64）和对象型（object）数据构成，其中对象型数据主要为城市、日期等信息。其中，Idx 作为唯一标识符，target 作为预测目标（标签），ListingInfo 记录了借款成交时间。

Userupdate_Info_*_Set（用户更新信息表）：该表记录了用户在借款成交前修改个人信息的历史行为。包含 372,463 条记录和 4 个字段。该表没有缺失值。

Idx：借款人标识符，用于与主表关联。
ListingInfo1：借款成交时间。
UserupdateInfo1：修改的具体字段名称。
UserupdateInfo2：修改行为发生的时间。

LogInfo_*_Set（用户行为日志表）：该表详细记录了用户在平台上的各种操作日志，如登录、浏览等。包含 580,551 条记录和 5 个字段。该表没有缺失值。

Idx：借款人标识符，用于与主表关联。
Listinginfo1：借款成交时间。
LogInfo1：操作代码。
LogInfo2：操作类别。
LogInfo3：操作发生的时间。

三个数据表均可通过 Idx 字段进行关联，从而将用户的静态信息与动态行为数据结合起来。

时间序列特征体现在 Userupdate_Info_*_Set 和 LogInfo_*_Set 表中均包含详细的时间戳信息，这为从时间维度提取特征（如操作频率、近期行为等）提供了可能性。ListingInfo 和 Listinginfo1 记录了借款成交时间，可以作为分析用户行为的基准时间点。

整体数据量较大，特别是用户行为日志表，记录非常详细，预示着特征工程将是本次实验的关键环节。从初步的 isnull().sum()检查来看，Userupdate_Info_*_Set 和 LogInfo_*_Set 表没有缺失值，但主表 Master_*_Set 存在较多缺失值（例如 WeblogInfo_1 等字段），在后续处理中需要进行填充或特殊处理。

三、 数据清洗与预处理（20%）

为了确保模型训练的稳定性和准确性，我们对原始的训练集和测试集进行了系统性的数据清洗与预处理。整个过程严格遵循“训练集与测试集同步处理”的原则，以防止数据泄露和分布不一致的问题。

1. 缺失值处理

首先统计每个字段的缺失情况：

```
WeblogInfo_1     0.967667
WeblogInfo_3     0.967667
UserInfo_12      0.630300
UserInfo_13      0.630300
UserInfo_11      0.630300
WeblogInfo_20    0.268333
WeblogInfo_21    0.102467
WeblogInfo_19    0.098767
WeblogInfo_2     0.055267
WeblogInfo_4     0.055033
WeblogInfo_5     0.055033
WeblogInfo_6     0.055033
UserInfo_2       0.010067
UserInfo_4       0.008933
WeblogInfo_32    0.008433
WeblogInfo_37    0.008433
WeblogInfo_35    0.008433
WeblogInfo_34    0.008433
WeblogInfo_33    0.008433
WeblogInfo_29    0.008433
dtype: float64
```

图 1 不同数据字段的缺失率

我根据不同特征的缺失率和业务含义，实施了多层次、有针对性的填充策略：

➢ **高缺失率特征剔除**

经计算，WeblogInfo_1 和 WeblogInfo_3 两个特征在训练集中的缺失率高达 96.8%。在如此高比例的缺失下，任何形式的填充都可能引入巨大的噪声和不确定性，且无法反映真实的数据分布。因此，我直接将这两个特征从训练集和测试集中彻底删除。

➢ **将“缺失”作为一种独立信息进行编码**

对象：

UserInfo_11、UserInfo_12、UserInfo_13（缺失率约 63%）
WeblogInfo_19（缺失率约 9.9%）
WeblogInfo_20（缺失率约 26.8%）
WeblogInfo_21（缺失率约 10.2%）

这些特征的缺失并非完全随机。例如，用户是否提供某些非必要信息本身就可能反映其谨慎程度或个人情况，这与违约风险可能存在关联。简单地用均值或众数填充会掩盖这种潜在的关联性。因此我将“缺失”视为一个独立的、有意义的类别。

对于 UserInfo_11、UserInfo_12、UserInfo_13，其原有值为 0.0 或 1.0，我将所有 NaN 值填充为新的类别 2.0。

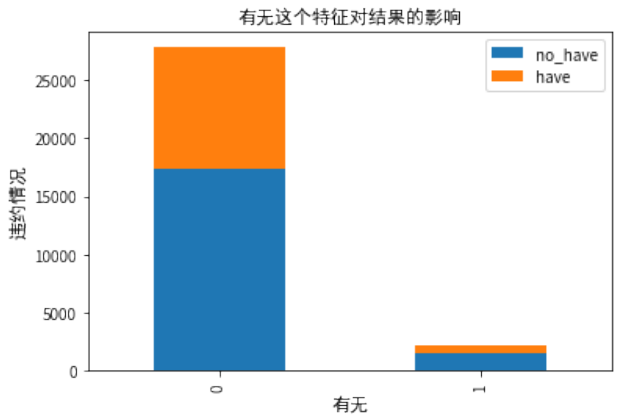

图 2 有无 UserInfo_11 对结果的影响

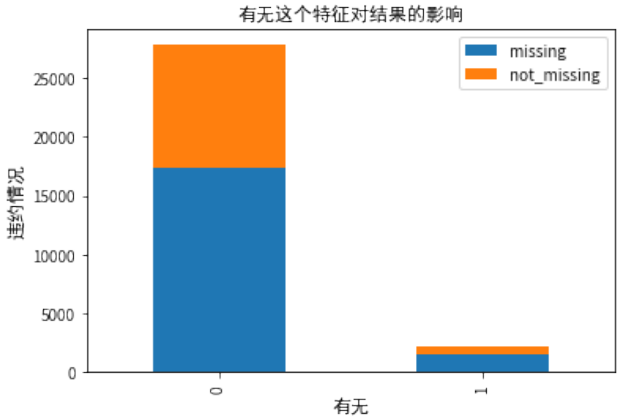

图 3 有无 UserInfo_12 对结果的影响

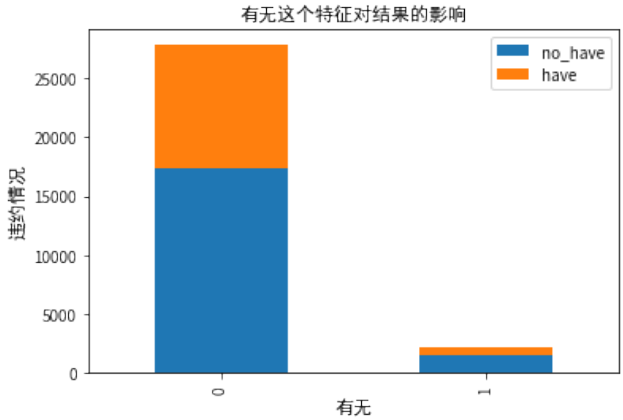

图 4 有无 UserInfo_13 对结果的影响

对于分类型变量 WeblogInfo_20 和 WeblogInfo_19，我们将 NaN 值填充为字符串“不详”。

```
array(['I', 'E', 'F', 'D', 'J', 'G', 'H', nan], dtype=object)
```

图 5 修改前 UserInfo_19 字段可能的内容

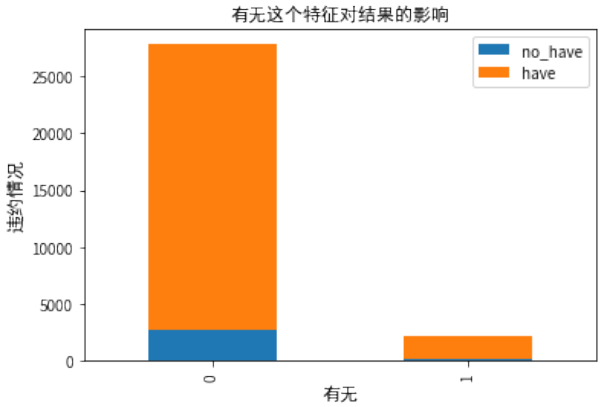

图 6 有无 UserInfo_19 对结果的影响

```
array(['I5', nan, 'I4', 'C19', 'I3', 'U', 'C21', 'C20', 'F13', 'C1',
       'C11', 'C12', 'C18', 'C15', 'O', 'C17', 'F12', 'F9', 'C16', 'I10',
       'C13', 'F11', 'F16', 'F7', 'C14', 'I6', 'F14', 'F10', 'C38', 'F15',
       'I11', 'I7', 'C39', 'F6', 'F3', 'C32', 'I8'], dtype=object)
```

图 7 修改前 UserInfo_20 字段可能的内容

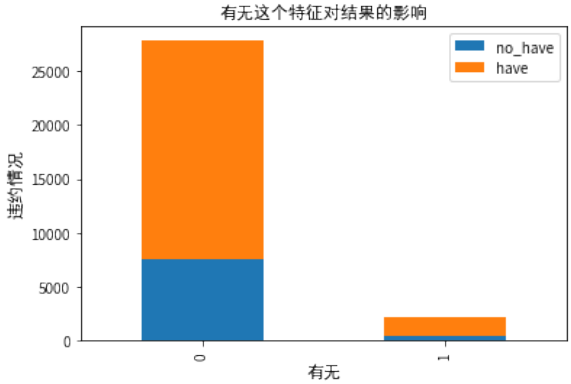

图 8 有无 UserInfo_20 对结果的影响

对于 WeblogInfo_21，其类别为字母，我将 NaN 填充为 0，以示区分。

```
array(['D', 'C', 'A', 'B', nan], dtype=object)
```

图 9 修改前 UserInfo_21 字段可能的内容

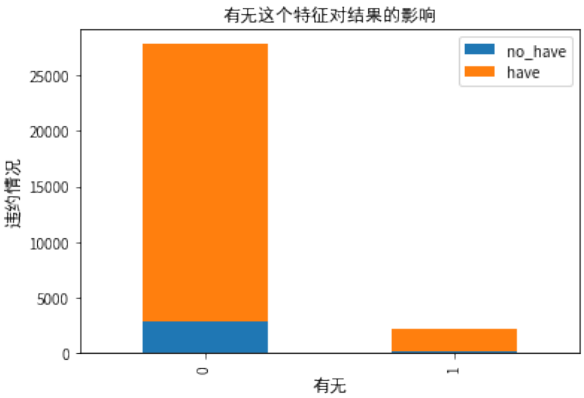

图 10 有无 UserInfo_21 对结果的影响

➢ **常规缺失值填充**

针对代码中剩余的、缺失率较低的特征，如 categoric_cols 列表内的所有特征（如 UserInfo_2 城市信息、UserInfo_9 运营商信息等），我们使用其在各自数据集（训练集或测试集）中的众数进行填充。这保证了填充值是该特征下最常见、最有可能的取值。

针对代码中 numeric_cols 列表内的所有数值型变量（如第三方征信评分、社交网络活跃度指标等），我们使用其在各自数据集中的均值进行填充。此方法可以在不引入极端值的情况下，保持特征整体的统计分布特性。

2. **离群点处理**

为增强模型的鲁棒性，我对数值型特征中的极端值进行了处理。该方法基于统计学经典的 3-sigma 准则进行离群点识别，并采用温和的替换策略避免信息损失。

处理方法: 我们没有直接删除包含离群点的样本，以避免信息损失。而是选择了一种更为温和的方式：将被识别出的离群点的值，替换为该特征的算术平均值。这既削弱了极端值对模型参数的过度影响，又保留了样本的其他信息。

对于数据集中的每个数值型特征$X = \{x_1, x_2, \dots, x_n\}$（其中 n 为样本数量），我们首先计算其统计量：

均值：$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$

标准差：$\sigma = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \mu)^2}$（使用样本标准差公式，以无偏估计）

根据 3σ准则，任何值x_i满足以下条件时，被判定为离群点：

$$x_i < \mu - 3\sigma \quad 或 \quad x_i > \mu + 3$$

假设数据近似正态，该区间覆盖了约 99.7%的数据分布，因此超出部分视为极端值。

3. **数据规范化与归一化**

为了消除因格式不一致导致的数据噪音，我们对特定字段进行了标准化处理：

➢ **文本格式统一**

UserInfo_9（运营商）该字段存在如 "中国移动 " 和 "中国移动" 这样因尾随空格导致的不一致。我们通过 strip() 函数清除了所有值的前后空格。

UserupdateInfo1（用户修改信息字段名）：该字段存在大小写混用，如 _EducationId 和_educationid。我通过 lower()函数将所有值转换为小写，确保它们被视为同一类别。

➢ **地理信息标准化**

对于 UserInfo_2、UserInfo_4、UserInfo_7、UserInfo_8、UserInfo_19、UserInfo_20

这些字段中存在不统一的行政区划后缀，例如“深圳市”与“深圳”、“广东省”与“广东”。我通过自定义的 encodingstr 和 encodingregion 函数，批量去除了字符串末尾的“市”、“省”、“自治区”等后缀，实现了地理名称的归一化，使得模型能够更准确地识别同一地区。

四、 特征工程（30%）

在数据清洗的基础上，我通过构建新特征和筛选现有特征，旨在最大化数据的价值，为模型提供更具预测力的输入。并且我进一步从未经利用的辅助信息表（Userupdate_Info_*_Set 和 LogInfo_*_Set）中深度挖掘用户行为模式，并对地理位置等高基数类别特征进行抽象化处理，构建了更为丰富和立体的特征体系。最后，我将所有特征进行整合，并实施最终的预处理步骤。

1 借款日期离散化

为了发现时间规律，我首先分析不同日期下，违约用户和正常用户的数量分布情况。

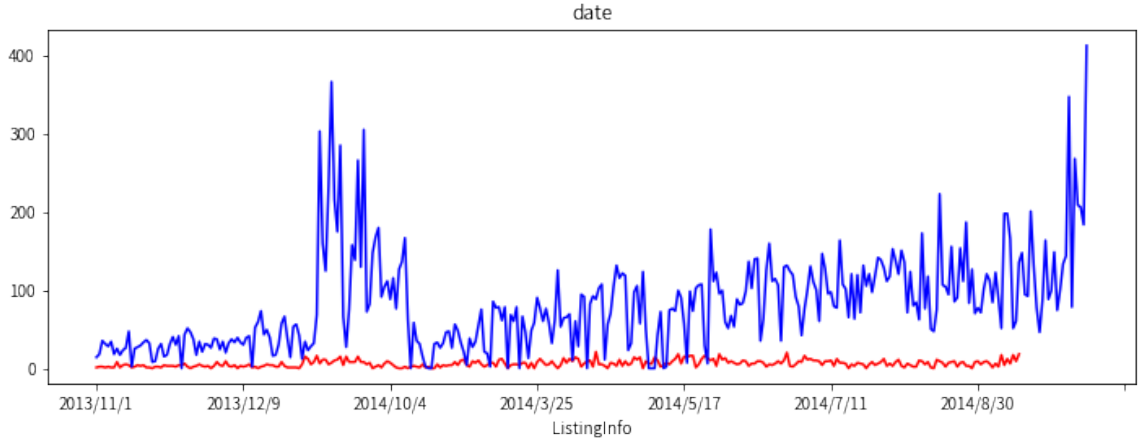

图 11 成交时间次数统计图

随后认识到 ListingInfo 字段的原始的日期格式（如"2014/3/5"）是字符串，无法直接用于大多数机器学习模型。更重要的是，日期中蕴含的周期性信息（如月份、星期）对预测用户行为可能至关重要。

所以我利用 pandas 的 DatetimeIndex 功能，从 ListingInfo 字段中提取了两个新的数值型特征：

month：借款成交的月份（例如，3）。

Day：借款成交的日期（例如，5）。

这两个新特征将时间的周期性信息显式化，便于树模型等算法发现潜在规律（例如，用户是否倾向于在月初或月末借款）。在衍生出新特征后，原始的 ListingInfo 字段因信息冗余而被删除。

2. 低方差过滤

为了识别并剔除那些在数据集中几乎没有变化（即方差或标准差极低）的特征。这些特征由于其取值高度一致，无法为区分违约与不违约用户提供有效信息，反而会增加模型训练的计算负担和噪声。

于是我首先筛选出所有的数值型特征列（numeric_cols），排除了 Idx、target、ListingInfo 等非输入特征。然后，计算了这些数值型特征在训练集上的标准差，并按升序排列。标准差是衡量数据离散程度最直观的指标。

```
WeblogInfo_49       0.000000
WeblogInfo_10       0.000000
WeblogInfo_44       0.015274
WeblogInfo_41       0.019996
WeblogInfo_46       0.028858
WeblogInfo_55       0.036507
SocialNetwork_1     0.036948
WeblogInfo_43       0.039138
WeblogInfo_47       0.042000
WeblogInfo_52       0.051930
SocialNetwork_11    0.052911
WeblogInfo_58       0.063455
WeblogInfo_40       0.070200
WeblogInfo_32       0.078750
WeblogInfo_31       0.083402
WeblogInfo_23       0.087106
WeblogInfo_35       0.104504
WeblogInfo_37       0.113071
WeblogInfo_54       0.115740
WeblogInfo_26       0.118533
dtype: float64
```

图 12 数值型特征在训练集上的标准差

分析代码输出的 feature_std.head(20)结果，我得到了关键发现：

WeblogInfo_49 和 WeblogInfo_10 这两个特征的标准差精确为 0.000000。这明确表明，在整个训练数据集中，这两个特征的取值是几乎完全恒定的。作为常量，它们对任何模型的预测都没有任何贡献。紧随其后的是 WeblogInfo_44（std=0.015）、WeblogInfo_41（std=0.020）、SocialNetwork_1（std=0.037）等一系列标准差极低的特征，它们虽然不是完全恒定，但变化范围非常小，提供的信息量也极其有限。

基于上述分析，我设定了一个严格的标准差阈值 threshold = 0.001。所有标准差低于此阈值的特征都被识别为低方差特征并被标记为待删除。根据计算结果，WeblogInfo_49 和 WeblogInfo_10 被纳入删除列表 features_to_drop。

我严格地从训练集和测试集中同步删除了这些特征。这一同步操作是至关重要的，它保证了模型在训练和预测时所见到的数据结构完全一致。通过低方差过滤，我成功地精简了特征空间，剔除了最无信息的数值型变量，为后续更复杂的特征选择和模型训练减轻了负担，并降低了引入无关噪声的风险。

3. 城市等级映射

目的是将高基数的地理位置文本信息（如城市名称），转化为低基数的、蕴含经济学意义的有序数值特征。直接对几百个城市进行独热编码会造成维度灾难和特征稀疏性问题。

我定义了一个 get_city_level 函数，该函数依据公认的城市分级标准，将具体的城市名称（如“深圳”、“成都”、“石家庄”）映射到其对应的经济等级（1-6 级）。其中，一线城市为 1 级，新一线为 2 级，以此类推，“不详”的城市被单独归为第 6 级。

```
first_level = ['北京', '上海', '广州', '深圳']
new_first_level = ['成都', '重庆', '杭州', '武汉', '西安', '郑州', '青岛', '
长沙', '天津', '苏州', '南京', '东莞', '沈阳', '合肥', '佛山']
second_level = ['昆明', '福州', '无锡', '厦门', '哈尔滨', '长春', '南昌', '济
南', '宁波', '大连', '贵阳', '温州', '石家庄', '泉州', '南宁', '金华', '常州',
'珠海', '惠州', '嘉兴', '南通', '中山', '保定', '兰州', '台州', '徐州', '太原
', '绍兴', '烟台', '廊坊']
third_level = ['海口', '汕头', '潍坛', '扬州', '洛阳', '乌鲁木齐', '临沂', '
唐山', '镇江', '盐城', '湖州', '赣州', '漳州', '揭阳', '江门', '桂林', '邯郸',
'泰州', '济宁', '呼和浩特', '咸阳', '芜湖', '三亚', '阜阳', '淮安', '遵义', '
银川', '衡阳', '上饶', '柳州', '淄博', '莆田', '绵阳', '湛江', '商丘', '宜昌',
'沧州', '连云港', '南阳', '蚌埠', '驻马店', '滁州', '邢台', '潮州', '秦皇岛',
'肇庆', '荆州', '周口', '马鞍山', '清远', '宿州', '威海', '九江', '新乡', '信
阳', '襄阳', '岳阳', '安庆', '菏泽', '宜春', '黄冈', '泰安', '宿迁', '株洲', '
宁德', '鞍山', '南充', '六安', '大庆', '舟山']
fourth_level = ['常德', '渭南', '孝感', '丽水', '运城', '德州', '张家口', '鄂
尔多斯', '阳江', '泸州', '丹东', '曲靖', '乐山', '许昌', '湘潭', '晋中', '安阳
', '齐齐哈尔', '北海', '宝鸡', '抚州', '景德镇', '延安', '三明', '抚顺', '亳州
', '日照', '西宁', '衢州', '拉萨', '淮北', '焦作', '平顶山', '滨州', '吉安', '
濮阳', '眉山', '池州', '荆门', '铜仁', '长治', '衡水', '铜陵', '承德', '达州',
'邵阳', '德阳', '龙岩', '南平', '淮南', '黄石', '营口', '东营', '吉林', '韶关
', '枣庄', '包头', '怀化', '宣城', '临汾', '聊城', '梅州', '盘锦', '锦州', '榆
林', '玉林', '十堰', '汕尾', '咸宁', '宜宾', '永州', '益阳', '黔南州', '黔东南
', '恩施', '红河', '大理', '大同', '鄂州', '忻州', '吕梁', '黄山', '开封', '郴
州', '茂名', '漯河', '葫芦岛', '河源', '娄底', '延边']
```

此函数被应用于 Master_*_Set 和 Master_Test_Set 表中的所有城市相关字段：UserInfo_2, UserInfo_4, UserInfo_8, UserInfo_20。

这种方法成功地将数百个离散的城市类别压缩为 6 个有序的数值等级，极大地降低了特征维度。更重要的是，它为模型引入了具有明确业务含义的宏观经济信息，因为不同城市等级的居民在收入水平、消费能力和信用风险上通常存在系统性差异。对于在城市列表中未能匹配的城市，我使用该列的众数（最常见的城市等级）进行填充，以保证数据的完整性。

4. 从辅助信息表中提取动态特征

我从用户的动态行为日志中聚合生成了一系列关键特征，以量化用户的稳定性和活跃度。

从 Userupdate_Info_*_Set 表中，我为每个用户聚合生成了以下四个核心特征，以刻画其个人资料的维护行为：

- numbers（信息修改总次数）：用户在借款前修改个人资料的总条目数。反映了用户资料的整体变动量。
- categorys（信息修改字段种类数）：用户修改过的不同字段的数量。例如，多次修改电话号码只计为 1 类。此特征衡量了用户资料修改的广度。
- times（信息修改发生天数）：用户进行信息修改操作的独立天数。若一天内多次修改，只计为 1 次。此特征反映了用户修改行为的频率。
- dates（首次修改距成交天数）：用户首次修改信息的时间点与最终借款成交日期之间相差的天数。该特征可以捕捉用户是在借款前长期准备还是临时抱佛脚。

从 LogInfo_*_Set 表中，我们提取了反映用户平台交互习惯的系列特征：

- LoginDays（总登录天数）：用户在观测期内有过登录记录的独立天数总和。这是衡量用户活跃度和粘性的直接指标。
- AverageLoginInterval（平均登录间隔）：用户相邻两次登录行为之间的平均间隔天数。一个稳定且较短的登录间隔可能意味着用户是平台的忠实使用者。
- OpCount_*系列特征（各类操作计数）：我们对用户的每一种操作代码 LogInfo1 进行了透视计数。即将原始的长格式日志数据转换为宽格式，每一列（OpCount_3001、OpCount_5 等）代表一种具体的操作类型，其值为该用户执行此操作的总次数。这为每个用户构建了一个详细的行为画像，全面反映了其在平台上的操作偏好。

由于训练集和测试集中的用户操作代码种类可能不完全一致，我取了两者的交集，确保模型在训练和预测时面对的是完全相同的特征列。

5. 最终特征整合

我将上述从 Userupdate_Info_*_Set 和 LogInfo_*_Set 聚合生成的特征表通过 pd.merge(how='left', on='Idx')的方式与主表 train_all（由 Master_*_Set 处理而来）进行合并。左连接操作可能导致部分在主表中存在、但在日志表中无记录的用户产生缺失值。这本身就代表了“无此行为”。

我对这些因合并产生的缺失值进行了二次填充：

AverageLoginInterval（平均登录间隔）：对于没有登录记录的用户，此项为 NaN，我们用全体用户的平均间隔进行填充。

LoginDays、times、categorys、numbers、dates 及所有 OpCount_*特征：对于这些特征，缺失即代表“0 次”，因此我用 0 进行填充，这符合其业务逻辑。

随后我进行了数值特征标准化，特征集中包含了不同量纲的数值型变量（例如，天数、次数、评分等）。为消除量纲差异对某些机器学习算法（如逻辑回归、树模型等）的影响，并有助于模型更快、更稳定地收敛，我们对数值特征进行了标准化处

理。

我采用 StandardScaler（Z-score 标准化）。该方法将每个数值特征的值转换为均值为 0、标准差为 1 的分布，使得所有数值特征都处于同一可比的尺度上。此操作在训练集上 fit_transform，在测试集上仅 transform，以防止测试集信息泄露。

经过以上详尽的特征工程步骤，我最终构建了一个高维、信息丰富的特征矩阵 train_all，并将其与目标变量 y_train 分离，为后续的模型训练做好了充分的准备。

五、 建模与优化（30%）

1. 建模前的通用数据准备

在将经过特征工程处理后的数据投入任何模型之前，我们执行了两个标准化的关键步骤，以确保数据的最终可用性和一致性。这两个步骤适用于后续所有模型的训练流程。

- **特征对齐**

在特征工程阶段，特别是涉及到基于数据内容的操作（如对操作代码进行透视计数）时，训练集和测试集可能因为数据本身的差异而产生不完全一致的特征列。例如，某个操作代码可能仅在训练集中出现，导致测试集缺少对应的 OpCount_*列。这会破坏模型的输入一致性，引发维度不匹配错误。

为了解决这个问题，我实现了一个 align_features 函数。该函数的核心任务是找出训练集和测试集共有的特征列，然后仅保留这个交集。具体而言，如果训练集特征列集为$\mathcal{F}_{train}$，测试集为$\mathcal{F}_{test}$，则保留$\mathcal{F}_{common} = \mathcal{F}_{train} \cap \mathcal{F}_{test}$。这保证了模型在训练和预测时面对的是完全相同的特征空间，从而避免了因特征不匹配导致的错误。

$$X_{train}^{aligned} = X_{train}[:, \mathcal{F}_{common}], \qquad X_{test}^{aligned} = X_{test}[:, \mathcal{F}_{common}]$$

- **最终类别特征编码**

尽管大部分特征已在前期处理为数值型，但数据集中仍可能残留部分 object 类型的列。逻辑回归等众多机器学习模型无法直接处理非数值型的输入，这会引发类型错误。所以我通过一个 encode_categorical 函数来完成最后的编码工作。该函数利用 sklearn.preprocessing.LabelEncoder 遍历数据集中所有剩余的 object 类型特征，并将它们的文本标签转换为唯一的整数。例如，一个包含$["A", "B", "A", "C"]$的列会被转换为$[0, 1, 0, 2]$。这一自动化步骤确保了所有输入到模型的数据都已是纯数值格式。编码过程可形式化为：

$$y_i = \text{LabelEncoder}(c_i), \qquad \forall c_i \in C$$

其中C是类别列的唯一值集，y_i是对应的整数编码。

2. 逻辑回归模型

我选择逻辑回归作为本次实验的第一个模型，它将作为后续更复杂模型的性能基

准。逻辑回归是经典的二分类算法，其结果稳定、可解释性强，是衡量模型性能的理想起点。并且模型训练速度快，计算资源消耗少，适合进行快速的初步验证。模型的系数可以直观反映不同特征对违约风险的影响，便于理解业务逻辑。

逻辑回归的核心是使用 Sigmoid 函数将线性组合映射到[0,1]概率区间，通过最大似然估计优化参数。假设特征为$X \in R^{n\times d}$，标签$y \in \{0,1\}^n$，模型预测 logit 为$z = X\theta$，则概率为：

$$p(y=1|x) = \sigma(z) = \frac{1}{1+e^{-z}}$$

损失函数为带正则化的负对数似然：

$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(p_i) + (1-y_i)\log(1-p_i)] + \frac{\lambda}{2}|\theta|_2^2$$

其中$p_i = \sigma(x_i^T\theta)$，控制 L2 正则化强度。优化通过梯度下降求解$\theta^* = \arg\min J(\theta)$，梯度为：

$$\nabla J(\theta) = \frac{1}{n}X^T(p-y) + \lambda$$

我们通过一个 train_logistic_regression 函数来封装模型的训练与预测流程，具体步骤如下：

在初始化过程中，我配置了以下核心参数：

- 采用 L2 正则化。这是一种标准的正则化技术，通过对模型系数的大小进行惩罚，有效防止模型在训练数据上产生过拟合，增强其在未知数据上的泛化能力。
- 选择 lbfgs 作为优化算法。该算法在处理中小型数据集时表现高效且稳健，能够确保模型参数找到最优解。
- 将求解器的最大迭代次数设置为 1000，确保在面对复杂或难以收敛的数据时，模型有足够的时间完成训练。
- 设置 n_jobs=-1 以利用所有可用的 CPU 核心进行并行计算，显著缩短训练所需时间。

调用模型的 fit()方法，将经过最终准备的训练数据 train_aligned 和对应的标签 y_train 作为输入，完成模型的拟合。通过梯度下降最小化上述损失函数$J(\theta)$。

训练完成后，使用训练好的模型 logistic_model 对同样经过处理的测试集 test_aligned 进行预测。

我调用 predict_proba(test_aligned)[:, 1]方法。该方法不直接输出类别（0 或 1），而是输出每个测试样本属于违约类别（类别 1）的概率$p_i = \sigma(X_i\theta)$。这个概率值可以被视为用户的“风险评分”，是评估模型性能和进行业务决策的关键依据。

最终，我将测试集用户的 Idx 与其对应的预测概率 yprob_logistic 合并，生成一个包含 ID 和 Prediction 两列的结果文件 result_logistic_regression.csv，作为本次逻

辑回归模型的输出。

3. 随机森林模型

随机森林通过构建多棵决策树并综合其预测结果，能够有效降低单一决策树的过拟合风险，具有良好的泛化能力。模型能够提供直观的特征重要性排序，有助于我们理解哪些特征对预测最具贡献。该模型对数据中的噪声和缺失值不敏感，且无需进行严格的特征缩放。

随机森林是 Bagging（Bootstrap Aggregating）与随机特征选择的集成方法。每棵决策树基于 Bootstrap 样本训练，分裂基于 Gini 不纯度或熵。单棵决策树的节点分裂选择最大化增益的特征 j 和阈值 s：

$$\text{Gain}(j,s) = \frac{N}{N}\left[I(p) - \frac{N_L}{N}I(p_L) - \frac{N_R}{N}I(p_R)\right]$$

其中$I(p) = -\sum_k p_k \log p_k$（熵）或 Gini $I(p) = 1 - \sum_k p_k^2$，p_L, p_R为左右子节点类别比例。随机森林的最终预测为 B 棵树的平均概率：

$$\hat{p}(x) = \frac{1}{B}\sum_{b=1}^{B} p_b(x)$$

其中$p_b(x)$是第 b 棵树的输出，特征重要性通过平均 Gini 减少量计算：

$$\text{Importance}(j) = \frac{1}{B}\sum_{b}\sum_{\text{splits on } j}\text{Gain}_{b,j}\cdot\frac{N_{\text{node}}}{N}$$

我从 sklearn.ensemble 导入 RandomForestClassifier 类并进行实例化。在初始化过程中，我配置了以下核心参数：

- 构建一个包含 00 棵决策树的森林，如上 B 为 500。
- 限制每棵决策树的最大深度为 11，防止过拟合。
- 在每次节点分裂时，随机选择$\sqrt{d}$个特征（d 为总特征数），增强随机性。
- 设置 n_jobs=-1 以利用所有可用的 CPU 核心进行并行计算，显著缩短训练所需时间。

调用模型的 fit()方法，在 train_aligned 数据上进行训练。每棵树独立训练，最终集成如上公式。

训练完成后，使用 predict_proba(test_aligned)[:, 1]方法生成测试集上每个用户的违约概率$\hat{p}(x)$。最终的预测结果被保存到 result_rf.csv 文件中。

4. LightGBM 模型

LightGBM（Light Gradient Boosting Machine）是一个基于梯度提升决策树（GBDT）的框架，以其极高的训练速度和低内存占用而闻名，尤其适合处理大规模数据集。它采用直方图算法（histogram-based）来加速分裂点查找，并支持叶子优

先生长（leaf-wise）策略，这使得它在处理高维稀疏数据时特别高效。

LightGBM 的核心在于梯度提升框架（Gradient Boosting Decision Tree, GBDT），其数学基础是迭代地添加弱学习器（决策树）来最小化损失函数。假设初始预测为 $F_0(x) = 0$，则在第 m 轮迭代中，模型通过拟合当前残差（负梯度）来更新预测。

梯度提升迭代公式：

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x), m = 1,2,\dots,M$$

- $F_m(x)$：第 m 轮的累积预测函数。
- $F_{m-1}(x)$：前一轮的预测。
- η：学习率（learning_rate=0.02），用于控制每棵树的贡献，避免步长过大导致震荡。
- $h_m(x)$：第 m 棵决策树，作为弱学习器，拟合于损失函数的负梯度$g_i = -\frac{\partial l(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)}$，其中$l$是二元交叉熵损失（对于二分类）：
- $l(y, F) = -\left[y \log\left(\sigma(F)\right) + (1-y)\log\left(1-\sigma(F)\right)\right]$
- $\sigma(F) = \frac{1}{1+e^{-F}}$是 Sigmoid 函数，最终输出概率$p = \sigma\left(F_M(x)\right)$。

树分裂增益计算：

LightGBM 使用直方图来近似梯度分布，提高效率。每个叶子的增益（Gain）计算为

$$Gain = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma$$

- G_L, G_R：左右子节点的一阶梯度累加（$G = \sum g_i$）。
- H_L, H_R：左右子节点的二阶梯度累加（Hessian，$H = \sum h_i = \sum \frac{\partial^2 l}{\partial F^2}$，用于二阶泰勒展开近似损失）。
- λ：L2 正则化系数（reg_lambda=0.45），γ：叶子分裂惩罚（隐含在 num_leaves=40 和 max_depth=11 中控制复杂度）。

此公式确保每一次分裂都能最大化损失减少，同时通过正则化防止过拟合。

正则化与随机性：

L1 正则化（reg_alpha=0.45）应用于叶子权重：

$$\Omega(h) = \alpha \sum |w_j| + \frac{1}{2}\lambda \sum w_j^2 + \gamma T$$

其中 T 是叶子数，w_j是叶子分数。

feature_fraction=0.2 引入特征子采样，类似于随机森林的随机性，进一步提升泛化。

LightGBM 有其自有的高效数据结构 lgb.Dataset。我们首先将训练数据和标签

封装成 lgb_train 对象，支持类别特征的原生处理。

在初始化过程中，我配置了以下核心参数：

- 这是一个二分类问题，使用交叉熵损失。
- 选择 gbdt 作为优化算法,使用传统的梯度提升决策树算法。
- 学习率设为 0.02，较小值促进稳定收敛。
- num_leaves=40、max_depth=11 控制单树复杂度，num_leaves 限制叶子数以避免过度生长。
- 每次迭代随机选 20%特征，增强随机性。
- L1 和 L2 正则化系数为 0.45，直接融入$\Omega(h)$。

模型训练与预测：

调用 lgb.train()函数，传入参数和数据，进行模型训练。通过早停（early stopping）和验证集监控 AUC 避免过拟合。使用训练好的 lgbmodel 对测试集 test_aligned 调用 predict() 函数，直接生成预测的违约概率$p = \sigma\left(F_M(x)\right)$。预测结果被保存到 result_lgb.csv 文件中。

5. XGBoost 模型

XGBoost（eXtreme Gradient Boosting）是另一个 GBDT 的实现，它引入了精确的正则化和并行分裂查找，支持 GPU 加速。在目标函数中直接包含了 L1L2 正则化项，有助于控制模型复杂度，避免噪声拟合。模型能够高效地进行并行计算，提升训练速度，尤其在特征分裂时。

XGBoost 的数学核心同样基于 GBDT，但强调二阶泰勒展开的精确近似和结构化正则化。其目标是最小化带正则化的损失函数，迭代添加树来修正残差。

整体目标函数：

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l\left(y_i, \widehat{y_i^{(t-1)}} + f_t(x_i)\right) + \Omega\left(f_t\right)$$

- $l(y, \hat{y})$：损失函数（如二元逻辑损失）。
- $f_t(x)$：第 t 棵树（弱学习器）。
- $\Omega(f_t)$：正则化项，定义为：

$$\Omega\left(f\right) = \gamma T + \frac{1}{2}\lambda|w|^2 + \alpha|w|_1$$

其中 T 是叶子数，w 是叶子权重向量，γ惩罚复杂树，λ、α分别为 L2 和 L1 正则化。

XGBoost 使用二阶展开来近似损失：

$$\mathcal{L}^{(t)} \simeq \sum_{i=1}^{n}\left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2\right] + \Omega(f_t)$$

- $g_i = \frac{\partial l\left(y_i, \widehat{y_i^{(t-1)}}\right)}{\partial \widehat{y_i^{(t-1)}}}$：一阶梯度（残差）。
- $h_i = \frac{\partial^2 l\left(y_i, \widehat{y_i^{(t-1)}}\right)}{\partial \left(\widehat{y_i^{(t-1)}}\right)^2}$：二阶 Hessian，提供曲率信息，提高近似精度。

对于叶子 j，其得分$w_j^* = -\frac{G_j}{H_j+\lambda}$，其中$G_j = \sum_{i \in I_j} g_i$，$H_j = \sum_{i \in I_j} h_i$。

每个潜在分裂的增益计算为：

$$Gain = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

这与 LightGBM 类似，但 XGBoost 通过精确分裂或近似算法来并行计算最佳分裂点，确保全局最优。

迭代更新：

$$y_i^{(t)} = \widehat{y_i^{(t-1)}} + \eta f_t(x_i), t = 1, \dots, T$$

最终概率$p_i = \sigma\left(\widehat{y_i^{(T)}}\right)$，其中$\sigma$是 Sigmoid。

行采样比例为 0.8，引入 Bootstrap-like 随机性，列采样 colsample_bytree 为 0.8，类似于 feature_fraction。

XGBoost 使用其专属的 DMatrix 数据结构，它针对内存和计算速度进行了优化，支持缺失值处理。我们将训练数据和测试数据分别转换为 dtrain 和 dtest 对象。

在初始化过程中，我配置了以下核心参数：

- 二分类，使用逻辑损失输出概率。
- 选择 gbtree 作为优化算法，基于树的提升器。
- 学习率设为 0.02，较小值促进稳定收敛。
- num_leaves=40、max_depth=11 控制单树复杂度，num_leaves 限制叶子数以避免过度生长。
- 每次迭代随机选 20%特征，增强随机性。
- L1 和 L2 正则化系数为 0.45，直接融入$\Omega(h)$。

调用 xgb.train()函数进行模型训练，并设置 num_boost_round=500。支持交叉验证和早停。使用训练好的 xgbmodel 对测试数据 dtest 调用 predict()函数，生成预测的违约概率$p = \sigma\left(\widehat{y^{(T)}}\right)$。最终的预测结果被保存到 result_xgb.csv 文件中。

6. Stacking 集成模型

Stacking（堆叠集成）是一种通过元学习器组合多个基模型的集成方法。它通过将基模型的预测结果作为新特征输入到元模型中，从而学习基模型之间的最优组合关系，通常能获得比单一模型更好的泛化性能。我利用它融合 LightGBM 和 XGBoost。

在 Stacking 集成模型中，元模型作为核心组合器，通过逻辑回归算法学习基模型预测结果的最优线性组合权重。

设有 K 个基模型$M_1, M_2, \dots, M_K$，和一个元模型M_{meta}。

第一层（基模型层）预测：

对于每个样本，基模型产生预测向量：

$$\mathbf{P_i} = [M_1(x_i), M_2(x_i), \dots, M_K(x_i)] \in R^K$$

其中$\mathbf{P_i}$是基模型的预测结果（在本实现中，K=2，对应 LightGBM 和 XGBoost 的违约概率输出）。

第二层（元模型层）训练：

使用基模型的预测结果作为新特征，元模型学习最终预测：

$$\hat{y} = M_{\text{meta}}(\mathbf{P})$$

其中$\mathbf{P} = [\mathbf{P_1}, \mathbf{P_2}, \dots, \mathbf{P_n}]^T \in R^{n\times K}$是所有样本的基模型预测矩阵。元模型通过最小化损失函数$\mathcal{L}(M_{\text{meta}})$优化参数，例如二元交叉熵：

$$\mathcal{L}(w, b) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y_i}) + (1 - y_i)\log(1 - \hat{y_i})] + \frac{\lambda}{2}|w|_2^2$$

其中$\hat{y_i} = \sigma(w^T\mathbf{P_i} + b)$，$\sigma(z) = \frac{1}{1+e^{-z}}$是 Sigmoid 函数，$w \in R^K$是权重向量，b 是偏置，$\lambda$是 L2 正则化系数。

元模型以 LightGBM 和 XGBoost 的预测概率作为输入特征，利用 Sigmoid 函数映射到概率空间，通过最小化二元交叉熵损失函数来学习各基模型的贡献度。该逻辑回归元模型还引入 L2 正则化惩罚项，有效控制权重系数幅度，防止对特定基模型的过度依赖，从而在保持模型解释性的同时实现基模型优势的智能融合与泛化性能提升。

在 Stacking 模型的训练过程中，首先进行数据准备阶段，将原始训练数据划分为 5 折以支持交叉验证生成元特征矩阵 $\mathbf{P}_{train}$，同时保持测试集 test_aligned 不变以用于最终预测；随后进入基模型训练阶段，对每个基模型（如 LightGBM 和 XGBoost）执行 5 折交叉验证训练，并在每一折的验证集上生成预测概率$p_{ij} =\sigma igma\left(F_{M_j}(x_i)\right)$（其中$F_{M_j}$为基模型的 logit 输出），然后组合所有折的预测形成完整的元特征矩阵$\mathbf{P}_{train} \in R^{n\times 2}$；接下来是元模型训练阶段，利用$\mathbf{P}_{train}$和原始标签训练逻辑回归元模型，应用 L2 正则化来控制模型复杂度，并通过梯度下降优化交叉熵损失函数以得到最优参数w^*, b^*；最后在测试预测阶段，使用完整训练数据重新训练所有基模型，对测试集生成预测矩阵$\mathbf{P}_{test} \in R^{m\times 2}$（其中 m 为测试样本数），元模型

则基于此矩阵计算最终预测概率 $p_{\text{final}} = M_{\text{meta}}(\mathbf{P}_{test}) = \sigma(w^T\mathbf{P}_{test} + b)$，从而实现高效的集成预测。

7. 模型检验

我对上述模型进行了检验，主要有测试集 AUC 和平台测试得分两个检验指标，在训练上，我还用了 5-折交叉验证，将训练集 D 随机划分为 5 个互斥子集分别训练和检验。结果如下：

表 1 模型对比结果

模型	Score	Test AUC
逻辑回归	0.64848	0.657858
随机森林	0.70362	0.713536
XGBoost	0.74544	0.745765
LightGBM	0.75274	0.752469
5 折-XGBoost	0.75248	**0.755142**
5 折- LightGBM	**0.75562**	0.751736
Stacking	0.75545	0.749348

六、 结论和心得体会（8%）

通过本次信用贷款违约预测实验，我深刻体会到数据质量与特征工程在金融风控建模中的决定性作用。面对包含 400 多个维度的复杂数据集，系统性的数据清洗和预处理是模型成功的基石。特别是对高缺失率字段的识别与处理、将“缺失”本身作为有效信息的编码策略，以及从用户动态行为日志中挖掘稳定性和活跃度特征，这些步骤显著提升了模型的预测能力。

在模型选择与优化过程中，梯度提升树框架（LightGBM 和 XGBoost）展现出了明显优势，其 AUC 得分均超过 0.75，显著优于传统的逻辑回归和随机森林模型。这验证了树模型在捕捉复杂非线性关系和特征交互方面的强大能力。值得注意的是，Stacking 集成模型通过元学习器融合多个基模型的预测结果，也取得了较佳表现，说明异构模型的优势互补能够进一步提升泛化性能。其中，5 折- LightGBM 取得平台测试最高得分 0.75562，5 折-XGBoost 取得测试最高 AUC 0.755142,说明 K-折交叉验证的重要性。

本次实验也暴露了真实业务数据中的典型挑战：高维稀疏特征、样本不均衡、时间序列信息的有效利用等。这些痛点的解决不仅需要技术手段，更需要对业务逻辑的深入理解。未来可探索的方向包括深度学习方法对原始特征的端到端学习、更精细的时间序列特征工程，以及针对不平衡学习的采样策略优化，以期在金融风控这一重要领域实现更精准的风险评估。

七、 参考文献：（2%）

[1] 于晓虹, 楼文高. 基于随机森林的 P2P 网贷信用风险评价、预警与实证研究[J]. 金融理论与实践, 2016(2):6.

[2] GUOLIN K, QI M F, THOMAS F, et al. LightGBM: a highly efficient gradient boosting decision tree[C]//Advances in Neural Information Processing Systems. Long Beach, CA, USA. 2017.

[3] CHEN T Q, GUESTRIN C. XGBoost: a scalable tree boosting system[C]//Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. San Francisco, CA, USA. 2016.